

InnoProcure — 4-Day Build Plan

Roles, Must-Build Features, and Time-Permitting Extensions

This is the scoped-down version for a 4-person team building in 4 days. It is deliberately smaller than a full production design — every cut is named so you can defend it if asked.

1. Roles and Why Each Is Necessary

Four roles, down from a fuller 7-role design. Two merges were made to fit the timeline — each is a real trade-off, not a free simplification.

1.1 Government Officer

What they do: Creates the challenge (problem, outcome target, budget, timeline), and later reviews the pilot and approves procurement/scale-up.

Why necessary: Departments are the actual source of demand — without this role there's no problem to solve. In a full design this would be split into a Challenge Creator and a separate Procurement Approver, so the person who approves a purchase isn't the same person who framed the problem (avoids rubber-stamping their own work). For a 4-day build, this split is cut.

Trade-off accepted: self-approval risk exists in this version — acceptable for a demo, not for a real deployment.

1.2 Evaluator

What they do: Scores startup proposals against a rubric, shortlists candidates, and later checks whether the pilot actually delivered the promised outcome.

Why necessary: Government departments usually lack the domain expertise to judge novel tech themselves — someone needs to score proposals on a fixed, comparable rubric so the shortlist is defensible. In a full design, scoring and pilot-verification would be two separate people, since letting someone verify the outcome of the startup they personally picked undermines "independent" verification. **Trade-off accepted:** independence is lost in this version — the same person scores and verifies, which is fine for a demo but would be a real weak point in production.

1.3 Startup

What they do: Builds a profile, applies to challenges, runs the pilot if selected, submits milestone evidence, tracks payment status.

Why necessary: This is the actual beneficiary the whole mechanism exists to serve — without a startup-side role there's nothing to evaluate or pilot. No merge/cut here; this role is unchanged from the full design.

1.4 Admin

What they do: Creates and manages accounts, assigns roles.

Why necessary: Someone has to provision the other three roles. Kept intentionally minimal for the 4-day build — doesn't need a full dashboard, a seed script or one basic panel is enough.

Cut entirely for this build: Public Viewer as a separate role/login. Instead, add a simple "public view" toggle inside the main app that shows a read-only slice of challenge status — this avoids building real anonymous access in a timeframe that doesn't support it.

2. Must-Build Features (the short list — build these first, in this order of priority)

These are the minimum features needed for the demo to tell a complete, coherent story end-to-end. If you run out of time, cut from the bottom of this list, not the top.

1. **Role-based login** — 4 roles above, with route/permission gating. Foundational; nothing else works without it.
2. **Challenge creation form** — problem description, outcome target, budget, timeline (Government Officer).
3. **Challenge browse/list view** — visible to Startup and in the public-view toggle.
4. **Startup profile + application form** — basic fields (name, capability tags, past experience) plus a proposal submission tied to a challenge.
5. **Scoring + shortlist** — a simple form (score + comment) and a sorted list of applicants by score (Evaluator). No multi-evaluator averaging logic needed — one evaluator's score is enough for a demo.
6. **Pilot agreement view** — a static template auto-filled with challenge/startup data. No real legal clause-library logic — just enough to show "a contract exists at this stage."
7. **Milestone list + manual verification** — Evaluator marks a milestone complete/incomplete. No AI, no automated data checks.
8. **Payment status field** — a manually-updated status (Pending/Released) tied to a verified milestone. No real payment-system integration.
9. **Go/no-go decision + final status** — Government Officer marks a pilot Approved/Scaled or Rejected at the end.

10. **Public view toggle** — read-only version of challenge status, no login required.
11. **Basic activity log** — a simple table logging who did what and when (challenge created, score submitted, milestone verified, decision made). This is what lets you claim "transparency" in the demo without building a full audit system.

Explicitly out of scope for the must-build list — do not attempt these under time pressure: AI-assisted anything, GeM/DPIIT/PFMS/DigiLocker integration, escrow payments, cross-department replication, automated matching, conflict-of-interest checks, real notification system, analytics dashboards. All of these move to Section 3 below.

3. Extra Features (build only if time remains)

Ordered roughly by effort-to-impact for a demo — top of the list is worth doing first if you have spare hours, bottom is genuinely optional.

Feature	What it adds to the demo	Rough effort
Notifications (in-app only)	Officer/Startup see a badge when status changes — makes the demo feel alive without needing email/SMS infra	Low
AI-assisted problem-structuring wizard	Officer types a rough problem, wizard suggests the structured outcome-metric version — good visual "AI" moment for judges	Low–Medium (one LLM API call with a constrained prompt)
Simple matching suggestion	When a challenge is published, suggest 2–3 startups from seed data with matching tags	Low (tag overlap, no ML needed)
Basic analytics view for Admin	Counts: challenges open, pilots running, avg. score, payment status breakdown	Low (aggregate queries over data you already have)
Conflict-of-interest checkbox	Evaluator must tick "no prior relationship with this startup" before scoring — cheap way to show you've thought about integrity	Low

Risk flag on challenge	Auto-flag if data sensitivity = high and no cybersecurity clause is attached	Low–Medium
AI-assisted milestone anomaly flag	Flags if submitted milestone data looks inconsistent, before Evaluator reviews it	Medium (needs believable sample data to demo well)
Mock external integration badge	Show a "Synced with GeM" / "Synced with PFMS" badge using dummy JSON, to visually signal the real design intent without building the real integration	Low (cosmetic only — be upfront in the pitch that it's mocked)
Real GeM/DPIIT/PFMS/DigiLocker integration	Actual data sync	High — do not attempt in 4 days
Escrow-style payment holding	Stronger payment-trust story	High, and has a real policy dependency (Finance Ministry sign-off) beyond just engineering effort
Cross-department replication registry	Shows scale-up story across departments	Medium–High — only worth it if the core loop is fully solid with time to spare

How to use this table in the pitch: if a judge asks "what's next," point here directly — it shows you scoped deliberately rather than ran out of time by accident.